Shenkar College of Engineering and Design

Department of Software Engineering

System Programming (Course 3503820)

5786 - Summer Semester | Lecturer: Yitzhak Nudler

Design Document

Assignment 2 (Challenging Version): Multi-Threaded WAL Crash Recovery Engine

Student Names: Amit Lachmann & Noam Eilat

Student IDs: 207448267 & 322713025

Submission Date: 06-09-2026

GitHub Repository: https://github.com/ViRaX5/Sys-Calls-Ex-2.git

Source File: wal_recovery.c

1. Overview

This document describes the design and implementation logic of wal_recovery, a multi-threaded crash-recovery engine written in C. It simulates the recovery phase of a database's Write-Ahead Logging (WAL) system: given a log of transactions that were being processed when the system crashed, the program reconstructs the correct final state of the database by replaying committed work and undoing whatever was left incomplete.

1.1 Background: ACID and Write-Ahead Logging

A transactional database must uphold four guarantees, known as ACID: Atomicity (a transaction either fully applies or not at all), Consistency (valid states only), Isolation (concurrent transactions behave as if run serially), and Durability (once committed, changes survive a crash). This assignment focuses on restoring Atomicity and Durability after a crash, using the Write-Ahead Logging technique: every change is first appended to a log on disk before being considered final. If the system crashes, the log - not the in-memory state - is the source of truth, and a recovery engine replays it to reconstruct what should have survived.

1.2 Execution Flow, End to End

The program reads wal_input.txt and produces accounts.txt, in four phases:

1. Phase A - Analysis (sequential, main thread): Read the WAL file line by line, up to and including "=== CRASH ===" (lines after it are never processed). Every UPDATE line's NEW value is applied immediately to an in-memory page table, and also recorded into a growable log-history array. BEGIN/COMMIT/ABORT lines update a per-transaction state tracker.

2. Phase B - Identify rollback candidates: Scan the transaction-state tracker for every transaction still in the ACTIVE state - i.e. one that began but never reached COMMIT or ABORT before the crash. These, and only these, get undone.

3. Phase C - Parallel Undo (worker threads): Spawn one pthread per still-ACTIVE transaction. Each thread scans the shared log-history array from the end backward, acting only on entries belonging to its own transaction, and restores each one's OLD value.

4. Phase D - Join and output: The main thread waits for every worker to finish, destroys the mutex, sorts the final page table alphabetically by key, and writes accounts.txt in "KEY: VALUE" format.

1.3 Input/Output Contract

File	Role	Format
wal_input.txt	Input log of chronological transaction events.	One event per line: "TX <id> BEGIN", "TX <id> UPDATE <key> OLD:<v1> NEW:<v2>", "TX <id> COMMIT", "TX <id> ABORT", or the literal line "=== CRASH ===".
accounts.txt	Output: the fully recovered key-value state.	"<KEY>: <VALUE>", one page per line, sorted alphabetically by key.

1.4 Documented Design Assumptions

Two behaviors follow from a literal reading of the assignment spec and are stated here explicitly rather than left implicit:

- Aborted transactions are not undone. The spec defines recovery as undoing transactions that were ACTIVE and never reached COMMIT or ABORT. A transaction that reached ABORT before the crash is therefore not touched by this recovery engine - whatever value it last wrote during the forward pass simply stands. A production WAL engine would typically have already logged compensating writes by the time an ABORT is recorded; this simplified log format never shows that happening, so this implementation follows the spec's literal definition rather than adding unstated behavior.
- Two ACTIVE transactions modifying the same key is not specially ordered. If two different still-active transactions both wrote to the same key before the crash, this implementation's mutex guarantees each individual page update is applied atomically, but does not guarantee the two undos apply in the chronologically correct order relative to each other, since thread scheduling between independent worker threads is not otherwise coordinated. The input is assumed not to exercise this case; see Section 3.3 for further discussion.

2. Data Structures

Two categories of structure are used: the four structs suggested directly by the assignment PDF (used essentially as given), and two growable-array containers added to actually hold data whose size is not known ahead of time - the PDF's structs describe shapes, not how collections of them get built up while parsing.

2.1 Structures Given by the Assignment Specification

Structure	Purpose
TxState (enum)	One of TX_INACTIVE / TX_ACTIVE / TX_COMMITTED / TX_ABORTED - the lifecycle state of a single transaction at any point while parsing.
Page	One key/value pair in the recovered database (a fixed-size key buffer and an int value).
SharedDatabase	The full in-memory page table (a fixed-capacity array of Page, per the spec's suggested struct), plus the page count and the pthread_mutex_t that protects concurrent access to both during Phase C.
LogEntry	One recorded UPDATE event: the transaction that made it, the key, and both the OLD and NEW values - kept so it can be replayed backward during Undo.
ThreadArgs	The argument bundle passed to each Undo worker thread: which transaction it rolls back, plus shared pointers to the full log history and the database.

2.2 Structures Added Beyond the Specification, and Why

The specification's structs describe individual records, but two collections of unknown-in-advance size are needed to actually parse a WAL file: the full list of UPDATE events (for log_history), and the current state of every transaction ID seen. Two small growable-array types were added for this - the same pattern used for reading files of unknown length, growing capacity in the code.

Structure	Purpose	Why This Shape
LogEntry array (log)	Accumulates every UPDATE event during Phase A, providing log_history/log_size for every worker thread in Phase C.	A plain dynamic array (malloc + realloc-doubling) rather than a linked list: it needs to be handed to every worker thread as one contiguous block with a known length (log_history/log_size), which a linked list cannot provide directly, and elements are only ever appended (never removed/reordered) during Phase A - the simplest structure that satisfies both is a growable array.
TxRecord array (tx-state tracker)	Tracks the current TxState of every transaction ID seen so far, looked up by tx_id as BEGIN/COMMIT/ABORT lines are processed.	A linear-scan growable array rather than a hash map or an array indexed directly by tx_id: transaction IDs are not guaranteed to be small, contiguous, or start at 0, so direct indexing is not safe, and at the input sizes this assignment deals with an O(n) scan per lookup is negligible - a hash map would add real implementation complexity (hashing, collision handling) for no measurable benefit here.

3. Concurrency and Synchronization Design

3.1 How Active Transactions Are Divided Across Worker Threads

After Phase A finishes, the transaction-state tracker is scanned once (single-threaded) to collect every tx_id still in the TX_ACTIVE state. Exactly one worker thread is spawned per active transaction - not per log entry, not per page. Each thread receives its own ThreadArgs with a distinct target_tx_id, but with log_history, log_size, and the SharedDatabase pointer identical across every thread's ThreadArgs. This division is what makes the work parallelizable in principle: different active transactions' undo work is independent of each other by construction (each thread only ever acts on log entries matching its own tx_id), and the only point where two threads can genuinely interact is if they happen to touch the same underlying page - which is exactly the case the mutex exists to handle.

3.2 Critical Section Analysis

There is exactly one mutex in the entire program (SharedDatabase.lock), and its critical section is deliberately drawn around the full "look up the page, then write its value" sequence inside the worker thread - not around the write alone. Concretely, per matching log entry, a worker thread performs:

```
lock(db->lock)
    page_index = find_page_by_key(entry.key, db)   // linear scan
    if page_index found:
        db->pages[page_index].value = entry.old_val
unlock(db->lock)
```

Everything else a worker thread touches is deliberately left unlocked, and each case is justified separately below rather than locked defensively “just in case”:

- log_history / log_size: read-only for every thread, for the thread's entire lifetime. Phase A (the only writer) has already completed before any thread is created, so there is no writer to race against - concurrent unsynchronized reads of data nothing is writing are never a race.
- db->page_count and every Page.key: written only during Phase A, never written again once a worker thread starts (workers only ever update the .value of an existing page - the forward pass has already registered every key that will ever appear). The page-lookup step inside the critical section above therefore does not, strictly speaking, need the lock to be safe on its own. It is kept inside the lock anyway: with the lock covering the full find-then-write sequence, “only one thread is ever inside this block at a time” is true by construction and needs no further argument - relying instead on the cross-phase invariant above would make the correctness argument strictly more fragile for no benefit, since the lookup itself is cheap.

3.3 Proof: No Race Conditions

The only memory that is both shared across threads and mutated after Phase A begins is db->pages[i].value. Every write to it happens inside the single critical section shown above, and every read of a value about to be overwritten happens inside that same critical section (the lookup that immediately precedes the write). Since pthread_mutex_lock/unlock around a single mutex guarantees mutual exclusion - at most one thread executes the protected region at a time - two threads can never simultaneously read-modify-write the same Page, which is precisely the definition of the race this design must prevent. This holds regardless of how many worker threads exist or how the OS schedules them.

Caveat, stated explicitly rather than hidden: this proof establishes that each individual update to a page is atomic and internally consistent. It does not establish that the final value of a page touched by two different active transactions reflects any particular chronological ordering between those two transactions' undos - see the assumption in Section 1.4. That is a limitation of the undo ordering strategy, not a violation of mutual exclusion.

3.4 Proof: No Deadlocks

Deadlock requires a cycle of threads each holding a lock the next one is waiting for. With exactly one mutex in the entire program, that cycle cannot form: a thread would need to already hold the lock and then block trying to acquire it again (self-deadlock via recursive locking) for any deadlock to be possible at all. This does not happen here - the lock is acquired once, the critical section performs only non-blocking in-memory operations (an array scan and a value assignment, no I/O, no waiting on another thread, no recursive call back into a locking function), and it is released before the loop moves to the next log entry. No thread ever attempts to acquire a second lock while holding the first, because there is no second lock to acquire.

3.5 Thread Lifecycle Summary

Step	Called From	Notes
pthread_mutex_init	main, before Phase A	Must exist before any code that could touch it runs.
pthread_create (x N)	main, after Phase B	One call per active transaction; each passed a distinct ThreadArgs.
pthread_join (x N)	main, after all creates	Deliberately a separate loop from creation - joining thread 0 before creating thread 1 would serialize the work.
pthread_mutex_destroy	main, after all joins	Safe only once every thread has exited, so nothing could still be holding or waiting on it.

4. Error Handling and Edge Cases

Every system call, memory allocation, and pthread operation that can fail is checked explicitly. The general policy: on an unrecoverable error, report a descriptive message to stderr and exit, releasing whatever resources were already acquired at that point.

Case	Detection	Handling
Missing input file	open() returns -1	Print the reason (via strerror) to stderr; exit(1) - the literal exit code the spec requires for this case specifically.
malloc()/realloc() failure	Returned pointer is NULL	perror(); free whatever was already allocated; exit(EXIT_FAILURE).
pthread_mutex_init() failure	Non-zero return value	Report to stderr; exit(EXIT_FAILURE) before any thread-related state is created.
pthread_create() failure	Non-zero return value	Report to stderr (including which transaction's thread failed); exit(EXIT_FAILURE). Checked explicitly per the spec's edge-case requirement.
pthread_mutex_lock()/unlock() failure	Non-zero return value	Report to stderr; the worker thread returns immediately rather than proceeding without holding the lock it was relying on.
Malformed log line	sscanf() does not match the expected field count	Report to stderr and skip just that line - a single corrupt line does not abort the whole recovery.
Blank lines	Line is empty after trimming whitespace	Silently skipped, per the spec's explicit instruction to ignore blank lines.
Page table exceeds MAX_PAGES	page_count would exceed the fixed 1000-entry capacity	Report to stderr; exit(EXIT_FAILURE) - a fixed-size table is the specification's own suggested structure, so this is treated as an input exceeding a stated, documented limit rather than a case to silently truncate.
No "=== CRASH ===" marker found	End of file reached without seeing the marker	A warning is printed, and end-of-file is treated as the crash point - the file is still fully usable, just noted as not matching the expected format exactly.

5. Build Instructions

Compiled with strict warning flags plus pthread linking, producing zero warnings or errors:

```
gcc -Wall -Wextra -std=gnu11 -O2 -pthread wal_recovery.c -o wal_recovery
```

A Makefile automates this:

```
make          # builds the wal_recovery executable
make clean    # removes the compiled binary and any accounts.txt output
```

5.1 Repository Contents

File	Purpose
wal_recovery.c	Single C source file containing the full implementation.
Makefile	Builds wal_recovery.c into the wal_recovery executable.
wal_recovery	The compiled executable.
DESIGN.pdf	This document.

GitHub repository: [FILL IN: https://github.com/<user>/<repo>] - repository visibility is set to Public so the grader can access all files via the link alone.